

CARLA-Autaware-Bridge: Facilitating Autonomous Driving Research with a Unified Framework for Simulation and Module Development

Gemb Kaljavesi¹, Tobias Kerbl¹, Tobias Betz¹, Kirill Mitkovskii² and Frank Diermeyer¹

Abstract—Extensive testing is necessary to ensure the safety of autonomous driving modules. In addition to component tests, the safety assessment of individual modules also requires a holistic view at system level, which can be carried out efficiently with the help of simulation. Achieving seamless compatibility between a modular software stack and simulation is complex and poses a significant challenge for many researchers. To ensure testing at the system level with state-of-the-art AV software and simulation software, we have developed and analyzed a bridge connecting the CARLA simulator with the AV software Autaware Core/Universe. This publicly available bridge enables researchers to easily test their modules within the overall software. Our investigations show that an efficient and reliable communication system has been established. We provide the simulation bridge as open-source software at <https://github.com/TUMFTM/Carla-Autaware-Bridge>.

I. INTRODUCTION

In 2018, the World Health Organization (WHO) published a road safety report [1] indicating that over one million individuals die annually in road accidents. Automated Vehicles (AVs) can potentially decrease this figure [2]. To realize this potential and facilitate the widespread adoption of autonomous vehicles, they need extensive testing, presenting a significant challenge [3], [4]. In general, approaches to autonomous driving can be categorized as modular or end-to-end. Modular approaches involve software comprised of distinct, self-contained modules addressing specific tasks interconnected with each other. In autonomous driving, these modules include mapping, localization, perception, planning and control [5]. The complex interdependencies among modules pose a known issue in autonomous robotics literature [6], addressed by frameworks like the Robot Operating System (ROS) [7]. End-to-end approaches aim to treat these modules as a machine learning task, often with a simpler architecture compared to modular ones.

However, modular approaches are more widespread in both industry and research. Their modularity allows teams to focus on specific tasks and independently enhance the system, ensuring operational functionality with practical intermediate outputs [8]. A modular approach can be tested at both the component and system level. For instance, perception can be evaluated using human-annotated datasets



Fig. 1: CARLA on the left and Autaware Core/Universe on the right during a simulation.

[9], while planning can be tested through low-fidelity simulations, providing a simplified representation of the environment, the kinematics, and sensors [10]. However, interactions between components are not considered, and not all possible errors in AV software can be detected [11]–[13]. Hence, system-level tests are necessary, involving evaluating the system as a whole. This can be efficiently achieved through High-Fidelity (HF) simulations, which incorporate mathematical models of the environment, the AV, surrounding road users, and sensors.

However, testing self-developed modules within a comprehensive open-source HF simulation framework is currently not possible with state-of-the-art AV software, nor are detailed performance analyses available for older frameworks.

A. Related Work

There is an increasing number of publicly available software projects in the field of autonomous driving. Researchers and the automotive industry can profit at a large scale from these open-source projects, sharing their results and leveraging the potential of a drastically growing community [5].

1) *Open-Source Software for Autonomous Vehicles*: Currently, the most advanced modular open-source software stacks for autonomous driving are Autaware Core/Universe [14] and Apollo [15]. Autaware is developed and maintained by the Autaware Foundation [16]. Whereas its first version, Autaware.AI [17], is ROS1 based, the latest distribution, Autaware Core/Universe, is based on ROS2. Baidu is the company responsible for the development of Apollo. Its latest update, Apollo 9.0, was launched at the end of 2023 [15]. Besides AV software with modular architectures, there are

¹Gemb Kaljavesi, Tobias Kerbl, Tobias Betz and Frank Diermeyer are with the Institute of Automotive Technology at the Technical University of Munich (TUM), DE-85748 Garching, Germany. {gemb.kaljavesi, tobias.kerbl, tobias94.betz, diermeyer}@tum.de

²Kirill Mitkovskii was not supported by any organization. kirill.mitkovskii@gmail.com

numerous end-to-end approaches for autonomous driving, such as ReasonNet [18]. As this work focuses on modular AV software, these are not further considered. Due to the widespread usage of ROS within the research community and the strong focus on open-source, Autoware constitutes a good option for research purposes. Apollo, however, lacks proper documentation and focuses more on industrial-scale in-field testing and data collection.

2) High-Fidelity Simulators for Autonomous Vehicles:

Testing AVs with HF simulators complements real-world testing and enables a more precise prediction of the AV's performance in the real world. The HF simulation data must closely match the real-world data [19]. Important performance indicators for HF simulation frameworks are the number of available assets (e.g., maps, sensors, or vehicles), scenario design, execution capabilities, and compatibility with external systems [19], [20]. There are various commercial HF simulators available, and a lot of companies involved in AV development built up their own simulation frameworks, which are not publicly available [19]. However, there are also various open-source projects that aim to provide HF simulation for research purposes. In order to evaluate the feasibility of different open-source HF simulators for our purpose, a subset of criteria partially based on [19]–[21] is used. Discontinued projects, like the widespread LGSVL simulator [22], are not suitable since they do not undergo further development. Some simulators are tailored for specific use cases, while others lack of realism. Since a HF simulator for autonomous driving in real-world settings is required, these simulators are also not an option. Both CARLA [23] and AWSIM [24] fulfill these initial requirements. They include sensor models for LiDAR, Camera, IMU, GNSS, and vehicle status information. Whereas CARLA comes with several different vehicle models and maps, AWSIM only includes one of each. In general, CARLA outperforms AWSIM in terms of provided assets. Both provide a ROS interface for communication with external AV software. As AWSIM was developed specifically for Autoware Core/Universe, no further middleware is required to use AWSIM with the latest Autoware release.

For the end-of-life Autoware.AI distribution, options for closed-loop testing with CARLA or the LGSVL simulator were implemented [25], [26]. There is also a bridge available for the simulation of Apollo 8.0 in CARLA [27]. A more recent bridge links the latest Autoware Core/Universe distribution with CARLA [28]. However, to simulate multiple vehicles more easily, this bridge includes an extra communication layer, potentially affecting performance without adding value to system-level tests. Despite these developments, effectively utilizing CARLA's extensive assets for testing and developing the prominent Autoware Core/Universe distribution remains an unsolved challenge.

B. Contribution

To provide a valuable option of system-level testing in a HF simulation environment with extensive assets, especially for research purposes, this work introduces a bridge to link

two widely adopted open-source projects: CARLA as simulation environment and Autoware Core/Universe as state-of-the-art software under test. Further, experiments were conducted to show the feasibility of the framework, namely the CARLA-Autoware-Bridge, for closed-loop simulation at system-level.

The reminder of this work is structured as follows. Section II provides a detailed description of the proposed simulation framework. Section III explains the design of the conducted experiments to showcase the capabilities of the CARLA-Autoware-Bridge. Results obtained from the experiments are presented in Section IV. Section V derives conclusions from the work, which we further discuss to indicate future research directions.

II. UNIFIED SIMULATION FRAMEWORK

To accomplish a HF simulation in real-time and offer an easy-to-use framework, we defined requirements to be fulfilled by the CARLA-Autoware-Bridge.

Interoperability: The bridge should offer an interface between the latest releases of CARLA and Autoware Core/Universe.

Compatibility: CARLA and Autoware should both be provided with a consistent sensor and vehicle setup.

Efficiency: The bridge should have a low overall latency and minimal hardware requirements.

Adaptability: Sensor setup and vehicle model should be configurable in a fast and flexible manner.

A. Overview

Fig. 2 depicts the software framework for system-level simulation of Autoware Core/Universe using CARLA. The CARLA simulator, with some of its functions, is shown on the left. It can execute control commands through a simulated actuator, save and implement simulation settings (e.g., weather), manage and publish the simulation time, and generate diverse sensor data. Communication to and from the CARLA simulator occurs via TCP/IP.

The framework leverages the existing ROS-Bridge for the CARLA Simulator [29], which has been enhanced to meet the authors' requirements. The primary role of the ROS-Bridge is to facilitate communication between TCP/IP and ROS 2. It also provides the server with the next time step after receiving all expected sensor data, ensuring synchronization and reproducibility of the simulation.

The CARLA-Autoware-Bridge, developed as part of this work, links the AV software and the ROS-Bridge. It manages initial simulation settings, sensor configurations of the AV, and the AV itself. Additionally, it converts sensor information from CARLA if necessary. For efficiency, non-converted sensor data is remapped to the correct topics and sent directly from the ROS-Bridge to Autoware. Control data sent by Autoware is transmitted directly to the ROS-Bridge and converted into CARLA control commands using a controller developed as part of this work.

Furthermore, we created a vehicle model, an exemplary sensor kit, and a Lanelet2 map required for Autoware in this

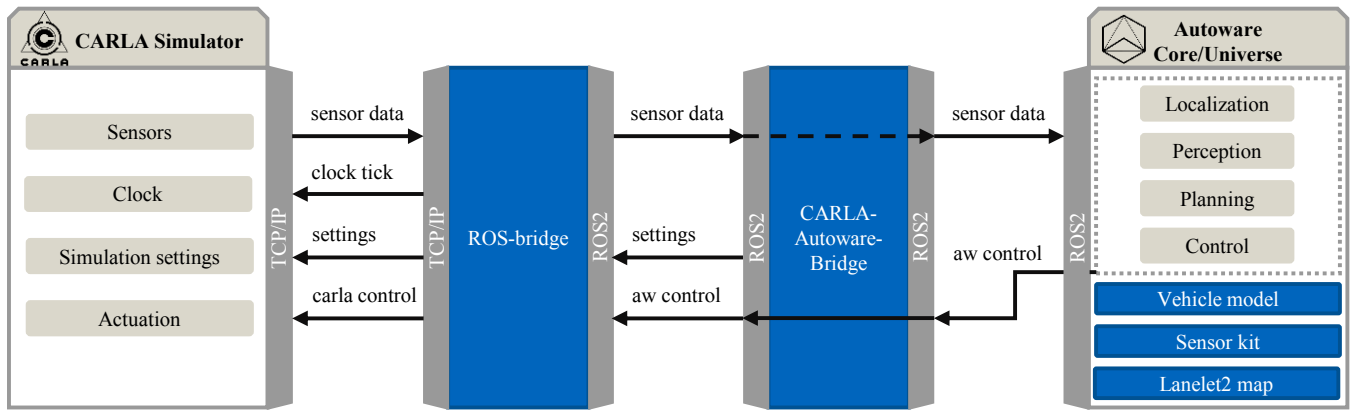


Fig. 2: Overview of the framework for system-level testing with the components developed or customized as part of in this work highlighted in blue.

context. This framework makes autonomous driving possible with a specific CARLA vehicle model, on one particular map, and with any sensor setup within the simulation.

B. CARLA to Autaware: Sensor Data

CARLA supports a variety of sensors, but its default data output is incompatible with Autaware Core/Universe. To optimize efficiency, the ROS-Bridge has been modified to handle and modify extensive data, such as camera images or LiDAR point clouds. As a result, that data does not need to pass through the CARLA-Autaware-Bridge. Adjustments made in either the ROS-Bridge or the CARLA-Autaware-Bridge include modifying the quality of service for messages and converting data projection from a left-handed coordinate system to a right-handed coordinate system. Additionally, certain information from CARLA needed summarization, while others required splitting. Moreover, ROS 2 messages had to be converted into messages suitable for Autaware Core/Universe.

C. Autaware to CARLA: Control Commands

The Autaware Core/Universe Controller transmits an Ackermann Control Command, encompassing target speed, target acceleration, and required tire angle, among other parameters. Autaware manages the vehicle's longitudinal movement through a Proportional-Integral-Derivative (PID) controller and the lateral movement using a Model Predictive Control (MPC). However, direct transfer of the Autaware Control Command to the CARLA simulation is not feasible. CARLA accepts the gas pedal position, brake pedal position, and steering wheel angle as inputs. To bridge this gap, we fine-tuned a PID controller to convert the longitudinal commands. The ROS-Bridge incorporates a simple PID controller implementation for this purpose. In addition to that, we generated a mapping to convert the tire angle to a steering angle. To enhance framework usability and ensure compatibility with future Autaware Core/Universe versions, controller parameter tuning for the Autaware PID or MPC was omitted.

D. Autaware Dependencies

In addition to ensuring compatible communication between Autaware and CARLA, providing the necessary dependencies for Autaware is crucial. Firstly, Autaware requires the simulated vehicle as a 3D model for rendering, along with the associated vehicle parameters. To achieve this, the VW T2 2021 was extracted from CARLA, converted into the correct format, and measured using the open-source tool Blender [30].

Additionally, Autaware requires the sensor kit, which provides information about the type and position of sensors within the simulation. For this purpose, an exemplary sensor kit was developed for the vehicle, serving as a template that can be expanded to include additional sensors.

Finally, Autaware needs an HD map of the CARLA map in Lanelet2 [31] format. The available CARLA maps, originally in OpenDrive format [32], were converted with the assistance of the CommonRoad Scenario Designer [33]. Furthermore, any missing or incorrect information in the maps was subsequently edited by hand.

III. EXPERIMENT DESIGN

We conducted experiments to demonstrate the capabilities of the CARLA-Autaware-Bridge and validate whether we satisfy the specified requirements. We used the latest version of Autaware Core/Universe for all experiments at the time of publication and CARLA 0.9.15. In the subsequent sections, we outline the design of these experiments. All the experiments are performed on a simulation workstation equipped with an Intel i9-14900K CPU (up to 6.0 GHz), 192 GB RAM, and an NVIDIA RTX 4090 GPU.

A. Transmission of Control Commands

We designed these experiments to evaluate our pipeline to transfer control commands from Autaware to CARLA. To enable a separate evaluation of both the introduced PID controller and the steering angle mapping, the experiment is divided into a longitudinal and lateral test case. This division allows for an isolated observation of the performance of

each introduced module. At the beginning of each test run, the vehicle is placed in the same initial pose on the map *Town10HD* and is assigned either the goal position for the longitudinal or lateral case.

The PID controller for longitudinal control is evaluated on a straight, with Autoware given the goal to drive along it with a constant target velocity of 8.33 m s^{-1} . We track the actual velocity of the vehicle and compare it against the given target velocity. The velocity curve is evaluated qualitatively regarding potential over- or undershooting and target value tracking. The experiment is conducted once with the default PID settings of the ROS-Bridge and once with manually tuned parameters obtained within this work.

The mapping of the steering angle for lateral control is assessed in a scenario in which the vehicle has to take three consecutive sharp turns. The experiment is performed once before the introduction of the mapping and afterward. To compare these two variants, we track the lateral deviation from the center line of the current lane.

B. Performance Benchmark

The core objective of our performance benchmark is to assess the efficiency across various configurations, focusing on the integration of multiple cameras and LiDAR sensors with different resolutions. This setup aims to simulate realistic autonomous driving scenarios, providing insights into the system's processing capabilities, communication efficiency, and overall performance under varying conditions. The benchmark is designed to cover different performance metrics, including CPU utilization and the latency of the bridge.

For analyzing the performance, we vary the amount of used LiDARs and cameras in CARLA. Ranging from 1 to 4 LiDARs (1 to 3 respectively for cameras), we additionally need to analyse the frames per seconds resulting in the CARLA simulation. Usually creating multiple instances of sensors increase the workload for the simulation environment. For the LiDAR sensor we investigate multiple point densities (1000 pt/s, 100 000 pt/s, 500 000, and 1 000 000 pt/s). For the camera setup we investigate two common resolution for images: 1280p x 720p (HD) and 1920p x 1080p (Full-HD).

The CPU usage is measured by acquiring the data for the individual processes launched on the compute system with standard Linux *ps*. We sample the process data every 200 ms. The processes are filtered to the relevant once for the ROS-Bridge and the CARLA-Autoware-Bridge.

To investigate the latency of the bridge, we use the framework developed by Betz et al. [34], which is based on *ros2_tracing* [35]. We measure the latency in the ROS-Bridge for the individual sensor setups with the corresponding ROS 2 callbacks. Note that most of the sensor data are not feed through the CARLA-Autoware-Bridge. Therefore, Autoware is directly subscribing to the modified ROS-Bridge.

All the experiments are conducted with the Autoware software running closed-loop. We developed a repeatable scenario according to [36] with a fixed start and end goal

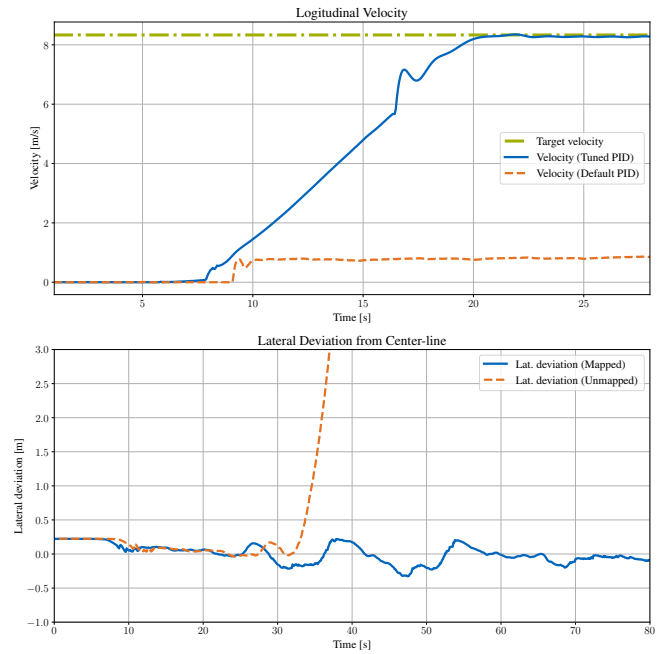


Fig. 3: Comparison of velocity and lateral deviation curves obtained from test runs with and without the introduced PID controller and steering angle conversion for control command transmission.

pose. The scenario is conducted in the map *Town10HD*. In addition to varying and analyzing the LiDAR and camera sensors, the other sensors in the sensor kit were consistently active throughout the test runs.

IV. RESULTS

The findings of the controller transmission analyses and results of the performance benchmarks are presented below.

A. Transmission of Control Commands

Figure 3 depicts the outcomes of the test runs conducted to assess the conversion and transmission of control commands from Autoware to CARLA. The data from the longitudinal test case reveal that the PID Controller from the ROS-Bridge, equipped with the default parameter set, cannot reach the target velocity. However, with the tuned parameter set, the vehicle achieves its intended target speed, even though the longitudinal velocity shows undesirable deviations in some sections.

The results from the test cases for the lateral control of the vehicle demonstrate that the lateral deviation from the lane center line is generally acceptable with the introduced mapping of the steering commands. The vehicle successfully navigated through all three sharp turns and stayed within the lane at all times. In contrast, the vehicle equipped with the direct forwarding of the steering angles departed from the lane at the first turn, whereby Autoware disengaged.

B. Performance Benchmark

As described in section III-B we analyse multiple performance metrics to give an insight into the computational

efficiency of the developed bridge. In Table I the results are depicted for the experiments regarding CPU utilization and the corresponding resulting FPS in the CARLA simulator by varying the sensor setup.

An increase in the point cloud density directly correlates with a decrease in FPS within CARLA. This outcome is anticipated, as higher point cloud densities demand more computational power to process the number of sensors. Interestingly, a decrease in FPS is accompanied by lower CPU utilization. This suggests that as the simulation becomes less efficient in terms of frame rate, it concurrently requires less computational effort. Furthermore, the addition of multiple sensors to the simulation impacts CPU usage less significant than increasing the point cloud density. This distinction implies that the computational load of processing high-density point clouds is more demanding than managing data from multiple sensors. The FPS drop of CARLA is higher for increased point cloud density as for multiple instances of sensors. For configurations involving a LiDAR setup with up to four sensors and a point cloud density of 100 000 pt/s, the simulation maintains a FPS rate above 30, indicating a threshold for maintaining real-time performance with acceptable limits. For the camera configurations it is more limited. While ensuring for both resolution a FPS above 30, we cannot add a second camera with Full-HD resolution without dropping the FPS below 30. Rendering camera images is a more resource extensive task compared to LiDAR point cloud generation. However, anomalies in CPU utilization were observed, with occasional slight increase compared to preceding configurations. For example with 3 LiDARs and 1000 pt/s a higher utilization is measured compared to the 2 LiDAR setup. These outliers suggest that there may be additional factors at play, influencing CPU demand in less predictable ways. The sensor kit released concurrently with this work, consisting of two LiDARs and one camera, utilized a single CPU core, demonstrating the ability to achieve an Framerate of 35 on the used simulation platform.

Fig. 4 shows the results for the latency metrics. It is observable that the latency increases with the number of points per second for the LiDAR and the camera resolution. The latency increases significantly, with larger point clouds. Although not depicted in the figure, the influence of multiple sensors was also analyzed. It was found that there is no significant impact on latency. The release sensor kit achieves a maximum latency of less than 15 ms and an average of 7.8 ms.

V. CONCLUSION

There is a strong demand for an open-source method to test a state-of-the-art modular AV software stack through HF simulation at the system level. We have developed a framework capable of facilitating such system-level testing using the CARLA Simulator and Autoware Core/Universe. We established the communication chain through an enhanced ROS-Bridge and our CARLA-Autoware-Bridge, also creating a compatible sensor kit, a vehicle model, and a

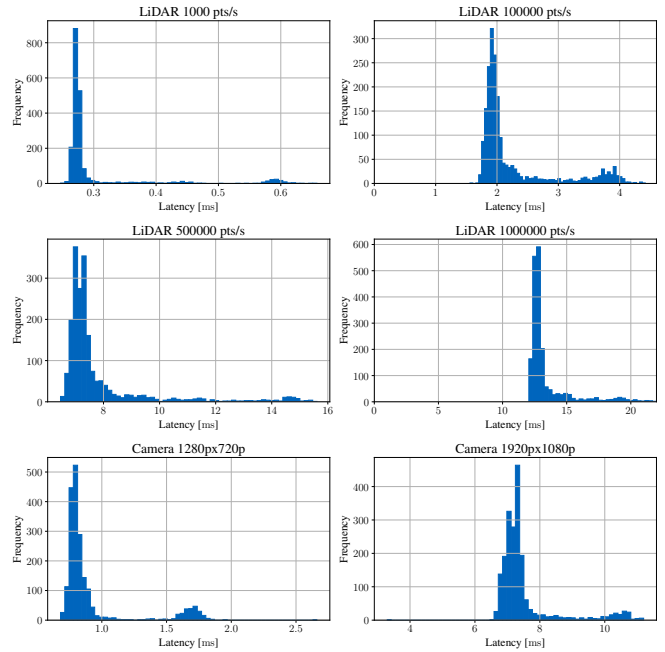


Fig. 4: Histograms displaying the latency of the developed framework, measuring the time between receiving messages from the simulation and outputting messages to the AV software for various sensors and sensor configurations.

Lanelet2 map. We provided and analyzed a conversion of Ackermann Control Commands to throttle, brake, and steering angle to control the actuators. A performance benchmark was employed to measure the latency and CPU utilization of the bridge, demonstrating its scalability. However, we observed that the bridge's latency increases rapidly with larger LiDAR point clouds, likely due to coordinate changes. This calculation can be implemented more efficiently in the future. Additionally, the map was manually generated; however, future efforts should include an automatic conversion from OpenDrive to Lanelet2 that is compatible with Autoware. In summary, an essential framework for system-level testing of autonomous driving functions has been developed. To facilitate enhanced testing, the system should be expanded in the future with a database of three-dimensional scenarios.

ACKNOWLEDGMENT

Gemb Kaljavesi and Tobias Kerbl, were the main developers of the presented work and main contributor of this paper. Tobias Betz contributed to the performance benchmarks. Kirill Mitkovskii helped with the development of the software. Frank Diermeyer made essential contributions to the conception of the research project and revised the paper critically for important intellectual content. This work was supported by the project Wies'n Shuttle (FKZ 03ZU1105AA) in the MCube cluster, the Federal Ministry of Economic Affairs and Climate Actions within the project ATLAS-L4 (FKZ 19A210481) and through basic research funds from the Institute for Automotive Technology.

TABLE I: The average CPU utilization and standard deviation, along with the frames per second of the simulation, are presented for various LiDAR-only and camera-only configurations, as well as the released sensor kit. (MCPU=Mean CPU Utilization, DCPU=CPU Utilization Standard Deviation, AFPS=Average Frames per Second).

Sensors	Sensor Count	1000 [pts/s]			100 000 [pts/s]			500 000 [pts/s]			1 000 000 [pts/s]								
		MCPU	DCPU	AFPS	MCPU	DCPU	AFPS	MCPU	DCPU	AFPS	MCPU	DCPU	AFPS						
LiDAR	1	135.6	0.10	101.2	132.3	2.06	95.7	109.0	1.00	52.6	98.9	1.65	36.3						
	2	125.57	3.18	96.9	120.9	3.06	78.9	92.7	2.08	35.2	90.5	1.08	21.4						
	3	127.9	1.18	87.5	117.3	2.00	61.1	93.7	1.85	25	84.4	1.22	15.1						
	4	117.0	3.01	69.7	119.4	1.31	50.6	84.5	1.74	19.1	83.5	1.09	11.2						
Sensors	Sensor Count	1280p x 720p			1920p x 1080p														
		MCPU	DCPU	AFPS	MCPU	DCPU	AFPS												
Camera	1	77.1	2.04	41.0	89.1	1.50	33.2												
	2	79.3	1.11	37.1	91.5	1.78	23.9												
	3	65.2	0.23	23.8	91.1	3.30	18.2												
Sensors	Sensor Count	Release																	
		MCPU	DCPU	AFPS															
LiDAR + camera	2 + 1	97.5	1.18	35.2	Release sensor kit: 500 000[pts/s]; 100 000[pts/s]; 1280px720p														

Release sensor kit: 500 000[pts/s]; 100 000[pts/s]; 1280px720p

REFERENCES

- [1] World Health Organization, *Global Status Report on Road Safety 2018*. Geneva: World Health Organization, 2018.
- [2] P. Tafiadis, A. Pirdavani, T. Brijs, and H. Farah, "Can Automated Vehicles Improve Cyclist Safety in Urban Areas?" *Safety*, vol. 5, no. 3, p. 57, Aug. 2019.
- [3] P. Koopman and M. Wagner, "Challenges in Autonomous Vehicle Testing and Validation," *SAE International Journal of Transportation Safety*, vol. 4, no. 1, pp. 15–24, Apr. 2016.
- [4] P. Koopman and M. Wagner, "Toward a Framework for Highly Automated Vehicle Safety Validation," in *WCX World Congress Experience*, Apr. 2018, pp. 2018–01–1071.
- [5] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda, "A Survey of Autonomous Driving: Common Practices and Emerging Technologies," *IEEE Access*, vol. 8, pp. 58 443–58 469, 2020.
- [6] S. Yang, X. Mao, S. Yang, Z. Liu, G. Chen, and S. Wang, "Towards a Robust Software Architecture for Autonomous Robot Software," in *Proceedings of 2017 the 7th International Workshop on Computer Science and Engineering*. WCSE, 2017.
- [7] M. Quigley *et al.*, "ROS: an open-source robot operating system," vol. 3, 01 2009.
- [8] A. Tampuu, T. Matiisen, M. Semikin, D. Fishman, and N. Muhammad, "A Survey of End-to-End Driving: Architectures and Training Methods," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 4, pp. 1364–1384, Apr. 2022.
- [9] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The KITTI dataset," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [10] M. Althoff, M. Koschi, and S. Manzing, "CommonRoad: Composable Benchmarks for Motion Planning on Roads," in *2017 IEEE Intelligent Vehicles Symposium (IV)*. Los Angeles, CA, USA: IEEE, Jun. 2017, pp. 719–726.
- [11] Z. Zhong, Y. Tang, Y. Zhou, V. d. O. Neves, Y. Liu, and B. Ray, "A Survey on Scenario-Based Testing for Automated Driving Systems in High-Fidelity Simulation," Dec. 2021.
- [12] J. Wang *et al.*, "AdvSim: Generating Safety-Critical Scenarios for Self-Driving Vehicles," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 9904–9913.
- [13] M. Malayjerdi, G. Kaljavesi, F. Diemeyer, and R. Sell, "Scenario-based Validation for Autonomous Vehicles with Different Fidelity Levels," in *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*, 2023, pp. 3411–3416.
- [14] "Autoware." [Online]. Available: <https://github.com/autowarefoundation/autoware>
- [15] "Apollo." [Online]. Available: <https://github.com/ApolloAuto/apollo>
- [16] "Autoware." [Online]. Available: <https://autoware.org/>
- [17] "Autoware.AI." [Online]. Available: <https://github.com/autowarefoundation/autoware.ai>
- [18] H. Shao, L. Wang, R. Chen, S. L. Waslander, H. Li, and Y. Liu, "ReasonNet: End-to-End Driving with Temporal and Global Reasoning," May 2023, arXiv:2305.10507 [cs]. [Online]. Available: <http://arxiv.org/abs/2305.10507>
- [19] Z. Zhong, Y. Tang, Y. Zhou, V. de Oliveira Neves, Y. Liu, and B. Ray, "A Survey on Scenario-Based Testing for Automated Driving Systems in High-Fidelity Simulation," *CoRR*, vol. abs/2112.00964, 2021. [Online]. Available: <https://arxiv.org/abs/2112.00964>
- [20] F. Rosique, P. J. Navarro, C. Fernández, and A. Padilla, "A Systematic Review of Perception System and Simulators for Autonomous Vehicles Research," *Sensors*, vol. 19, no. 3, p. 648, Jan. 2019. [Online]. Available: <https://www.mdpi.com/1424-8220/19/3/648>
- [21] W. Ding, C. Xu, M. Arief, H. Lin, B. Li, and D. Zhao, "A Survey on Safety-Critical Driving Scenario Generation—A Methodological Perspective," *IEEE Transactions on Intelligent Transportation Systems*, pp. 1–18, 2023.
- [22] G. Rong *et al.*, "LGSVL Simulator: A High Fidelity Simulator for Autonomous Driving," in *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, 2020, pp. 1–6.
- [23] A. Dosovitskiy, G. Ros, F. Codevilla, A. M. López, and V. Koltun, "CARLA: An Open Urban Driving Simulator," *CoRR*, vol. abs/1711.03938, 2017. [Online]. Available: <http://arxiv.org/abs/1711.03938>
- [24] "AWSIM." [Online]. Available: <https://github.com/tier4/AWSIM>
- [25] "Autoware.AI Simulation." [Online]. Available: https://github.com/autowarefoundation/autoware.ai_simulation
- [26] "CARLA-Autoware." [Online]. Available: <https://gitlab.com/hatem-darweesh-autoware.ai/carla-autoware>
- [27] "CARLA-Apollo-Bridge." [Online]. Available: https://github.com/guardstricklab/carla_apollo_bridge
- [28] "Zenoh-CARLA-Bridge." [Online]. Available: https://github.com/evshary/zenoh_carla_bridge
- [29] "CARLA-ROS-Bridge." [Online]. Available: <https://github.com/carla-simulator/ros-bridge>
- [30] "Blender." [Online]. Available: <https://github.com/blender>
- [31] F. Poggenhans *et al.*, "Lanelet2: A high-definition map framework for the future of automated driving," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 1672–1679.
- [32] M. Dupuis, M. Strobl, and H. Grezlikowski, "OpenDRIVE 2010 and Beyond—Status and Future of the de facto Standard for the Description of Road Networks," in *Proc. of the Driving Simulation Conference Europe*, 2010, pp. 231–242.
- [33] S. Maierhofer, M. Klischat, and M. Althoff, "CommonRoad Scenario Designer: An Open-Source Toolbox for Map Conversion and Scenario Creation for Autonomous Vehicles," in *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, 2021, pp. 3176–3182.
- [34] T. Betz, M. Schmeller, A. Korb, and J. Betz, "Latency Measurement for Autonomous Driving Software Using Data Flow Extraction," in *2023 IEEE Intelligent Vehicles Symposium (IV)*, 2023, pp. 1–8.
- [35] C. Bédard, I. Lütkebohle, and M. Dagenais, "ros2.tracing: Multipurpose Low-Overhead Framework for Real-Time Tracing of ROS 2," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6511–6518, 2022.
- [36] T. Betz, M. Schmeller, H. Teper, and J. Betz, "How Fast is My Software? Latency Evaluation for a ROS 2 Autonomous Driving Software," in *2023 IEEE Intelligent Vehicles Symposium (IV) (IEEE IV 2023)*. IEEE, 2023.